


```
|
|
|_ resources/
|   |_ application.properties
|
|_ test/
|   |_ java/
|       |_ com/
|           |_ example/
|               |_ spring/
|                   |_ ProductControllerTest.java
```

2. pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>

  <artifactId>spring-framework-7-annotation-sequence</artifactId>

  <version>1.0.0</version>

  <properties>

    <java.version>17</java.version>

    <spring.version>7.0.8</spring.version>

    <tomcat.version>11.0.10</tomcat.version>

    <maven.compiler.release>17</maven.compiler.release>

    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

  </properties>

  <dependencies>

    <!-- Spring Context -->
    <dependency>
```

```
<groupId>org.springframework</groupId>  
<artifactId>spring-context</artifactId>  
<version>${spring.version}</version>  
</dependency>
```

```
<!-- Spring MVC -->  
<dependency>  
  
    <groupId>org.springframework</groupId>  
  
    <artifactId>spring-webmvc</artifactId>  
  
    <version>${spring.version}</version>  
  
</dependency>
```

```
<!-- Spring Test -->  
<dependency>  
  
    <groupId>org.springframework</groupId>  
  
    <artifactId>spring-test</artifactId>  
  
    <version>${spring.version}</version>  
  
    <scope>test</scope>  
  
</dependency>
```

```
<!-- Jakarta Servlet -->  
<dependency>  
  
    <groupId>jakarta.servlet</groupId>  
  
    <artifactId>jakarta.servlet-api</artifactId>  
  
    <version>6.1.0</version>  
  
</dependency>
```

```
<!-- Embedded Tomcat -->  
<dependency>
```

```
<groupId>org.apache.tomcat.embed</groupId>  
<artifactId>tomcat-embed-core</artifactId>  
<version>${tomcat.version}</version>  
</dependency>
```

```
<dependency>  
  <groupId>org.apache.tomcat.embed</groupId>  
  <artifactId>tomcat-embed-jasper</artifactId>  
  <version>${tomcat.version}</version>  
</dependency>
```

```
<!-- Bean Validation -->  
<dependency>  
  <groupId>org.hibernate.validator</groupId>  
  <artifactId>hibernate-validator</artifactId>  
  <version>9.0.1.Final</version>  
</dependency>
```

```
<dependency>  
  <groupId>jakarta.el</groupId>  
  <artifactId>jakarta.el-api</artifactId>  
  <version>6.0.1</version>  
</dependency>
```

```
<dependency>  
  <groupId>org.glassfish.expressly</groupId>  
  <artifactId>expressly</artifactId>
```

```
        <version>6.0.0</version>
    </dependency>

    <!-- JUnit -->
    <dependency>

        <groupId>org.junit.jupiter</groupId>

        <artifactId>junit-jupiter</artifactId>

        <version>6.0.1</version>

        <scope>test</scope>
    </dependency>
</dependencies>

<build>

    <plugins>

        <!-- Java compiler -->
        <plugin>

            <groupId>org.apache.maven.plugins</groupId>

            <artifactId>maven-compiler-plugin</artifactId>

            <version>3.14.0</version>

            <configuration>

                <release>${maven.compiler.release}</release>

            </configuration>
        </plugin>

        <!-- Run main class -->
        <plugin>

            <groupId>org.codehaus.mojo</groupId>

            <artifactId>exec-maven-plugin</artifactId>
```

```
<version>3.5.0</version>

<configuration>

    <mainClass>
        com.example.spring.SpringApplication
    </mainClass>

</configuration>

</plugin>

</plugins>

</build>

</project>
```

3. application.properties

Location

```
src/main/resources/application.properties
app.name=Spring Framework Annotation Demo

app.version=1.0

server.port=8080
```

4. SpringApplication.java

Location

```
src/main/java/com/example/spring/SpringApplication.java
package com.example.spring;

import org.apache.catalina.startup.Tomcat;
import org.springframework.web.context.support.AnnotationConfigWebApplicationContext;
import org.springframework.web.servlet.DispatcherServlet;

public class SpringApplication {

    public static void main(String[] args) throws Exception {

        /*
```

```

    * Create Spring Web Application Context
    */
    AnnotationConfigWebApplicationContext context =
        new AnnotationConfigWebApplicationContext();

    /*
    * Register Java configuration class
    */
    context.register(AppConfig.class);

    /*
    * Create embedded Tomcat
    */
    Tomcat tomcat = new Tomcat();

    tomcat.setPort(8080);

    tomcat.getConnector();

    /*
    * Create Tomcat context
    */
    String contextPath = "";

    var tomcatContext =
        tomcat.addContext(
            contextPath,
            System.getProperty("java.io.tmpdir")
        );

    /*
    * Create DispatcherServlet
    */
    DispatcherServlet dispatcherServlet =
        new DispatcherServlet(context);

    /*
    * Register DispatcherServlet
    */
    Tomcat.addServlet(
        tomcatContext,
        "dispatcher",
        dispatcherServlet
    );

```

```

/*
 * Map DispatcherServlet
 */
tomcatContext.addServletMappingDecoded(
    "/",
    "dispatcher"
);

/*
 * Start server
 */
tomcat.start();

System.out.println(
    "Spring Framework application started."
);

System.out.println(
    "URL: http://localhost:8080/api/products"
);

/*
 * Keep server alive
 */
tomcat.getServer().await();
}
}

```

5. AppConfig.java

```

package com.example.spring;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.PropertySource;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;

@Configuration
@EnableWebMvc
@ComponentScan(basePackages = "com.example.spring")
@PropertySource("classpath:application.properties")
public class AppConfig {

}

```

Annotation sequence

```
@Configuration
↓
@EnableWebMvc
↓
@ComponentScan
↓
@PropertySource
```

@Configuration identifies a configuration class that Spring can process for bean definitions. [Home](#)

6. InfrastructureConfig.java

```
package com.example.spring.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Primary;
import org.springframework.context.annotation.Profile;
import org.springframework.core.task.TaskExecutor;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

@Configuration
public class InfrastructureConfig {

    @Bean
    @Primary
    public TaskExecutor taskExecutor() {

        ThreadPoolTaskExecutor executor =
            new ThreadPoolTaskExecutor();

        executor.setCorePoolSize(2);

        executor.setMaxPoolSize(4);

        executor.setQueueCapacity(20);

        executor.setThreadNamePrefix(
            "spring-demo-"
        );

        executor.initialize();

        return executor;
    }
}
```

```

    }

    @Bean
    @Profile("dev")
    public String devOnlyBean() {

        return "Development profile bean";
    }
}

```

@Bean is a method-level annotation used to register the returned object as a Spring-managed bean.
[Home](#)

7. Product.java

```

package com.example.spring.model;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Positive;

public record Product(

    long id,

    @NotBlank(
        message = "name is required"
    )
    String name,

    @Positive(
        message = "price must be positive"
    )
    double price
){
}

```

8. ProductRepository.java

```

package com.example.spring.repository;

import com.example.spring.model.Product;

import org.springframework.stereotype.Repository;

```

```
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicLong;
```

```
@Repository
```

```
public class ProductRepository {
```

```
    private final AtomicLong sequence =
        new AtomicLong(2);
```

```
    private final List<Product> products =
        new ArrayList<>{
            List.of(

                new Product(
                    1L,
                    "Laptop",
                    75000.0
                ),

                new Product(
                    2L,
                    "Monitor",
                    25000.0
                )
            )
        };
```

```
    public List<Product> findAll() {

        return List.copyOf(products);
    }
```

```
    public Product save(Product product) {

        Product saved =
            new Product(

                sequence.incrementAndGet(),

                product.name(),

                product.price()

            );
```

```

        products.add(saved);

        return saved;
    }

    public Product findById(long id) {

        return products
            .stream()

            .filter(
                product ->
                    product.id() == id
            )

            .findFirst()

            .orElseThrow(
                () ->
                    new ProductNotFoundException(id)
            );
    }

    public static class ProductNotFoundException
        extends RuntimeException {

        public ProductNotFoundException(long id) {

            super(
                "Product not found: " + id
            );
        }
    }
}

```

9. ProductService.java

```

package com.example.spring.service;

import com.example.spring.model.Product;
import com.example.spring.repository.ProductRepository;

import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.beans.factory.annotation.Value;

```

```
import org.springframework.stereotype.Service;
```

```
import java.util.List;
```

```
import java.util.concurrent.Executor;
```

```
@Service
```

```
public class ProductService {
```

```
    private final ProductRepository repository;
```

```
    private final Executor executor;
```

```
    private final String applicationName;
```

```
    public ProductService(
```

```
        ProductRepository repository,
```

```
        @Qualifier("taskExecutor")
```

```
        Executor executor,
```

```
        @Value("${app.name}")
```

```
        String applicationName
```

```
    ) {
```

```
        this.repository =  
            repository;
```

```
        this.executor =  
            executor;
```

```
        this.applicationName =  
            applicationName;
```

```
    }
```

```
    public List<Product> findAll() {
```

```
        return repository.findAll();  
    }
```

```
    public Product findById(long id) {
```

```
        return repository.findById(id);  
    }
```

```

public Product create(Product product) {

    executor.execute(

        () ->
            System.out.println(
                "Async audit from "
                + applicationName
            )

    );

    return repository.save(product);
}
}

```

10. ProductController.java

```

package com.example.spring.controller;

import com.example.spring.model.Product;
import com.example.spring.service.ProductService;

import jakarta.validation.Valid;

import org.springframework.http.HttpStatus;

import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService service;

    public ProductController(
        ProductService service
    ) {

        this.service = service;
    }

    @GetMapping
    public List<Product> findAll() {

```

```

        return service.findAll();
    }

    @GetMapping("/{id}")
    public Product findById(
        @PathVariable long id
    ) {

        return service.findById(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Product create(
        @Valid
        @RequestBody
        Product product
    ) {

        return service.create(product);
    }
}

```

11. GlobalExceptionHandler.java

```

package com.example.spring.controller;

import com.example.spring.repository.ProductRepository.ProductNotFoundException;

import org.springframework.http.HttpStatus;

import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(
        ProductNotFoundException.class
    )
    @ResponseStatus(
        HttpStatus.NOT_FOUND
    )

```

```

public Map<String, Object> handleNotFound(
    ProductNotFoundException ex
){
    return Map.of(
        "error",
        ex.getMessage(),
        "status",
        404
    );
}
}

```

12. ProductControllerTest.java

```

package com.example.spring;

import com.example.spring.controller.ProductController;
import org.junit.jupiter.api.Test;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import static org.junit.jupiter.api.Assertions.assertNotNull;

class ProductControllerTest {

    @Test
    void contextCreatesController() {

        try {
            var context =
                new AnnotationConfigApplicationContext(
                    AppConfig.class
                )
        } {

            assertNotNull(
                context.getBean(
                    ProductController.class
                )
            );
        }
    }
}

```

```
}  
}  
}
```

13. Run Spring Framework project

```
cd spring-framework-7-annotation-sequence
```

Then:

```
mvn clean test
```

Then:

```
mvn exec:java
```

Application:

```
http://localhost:8080/api/products
```

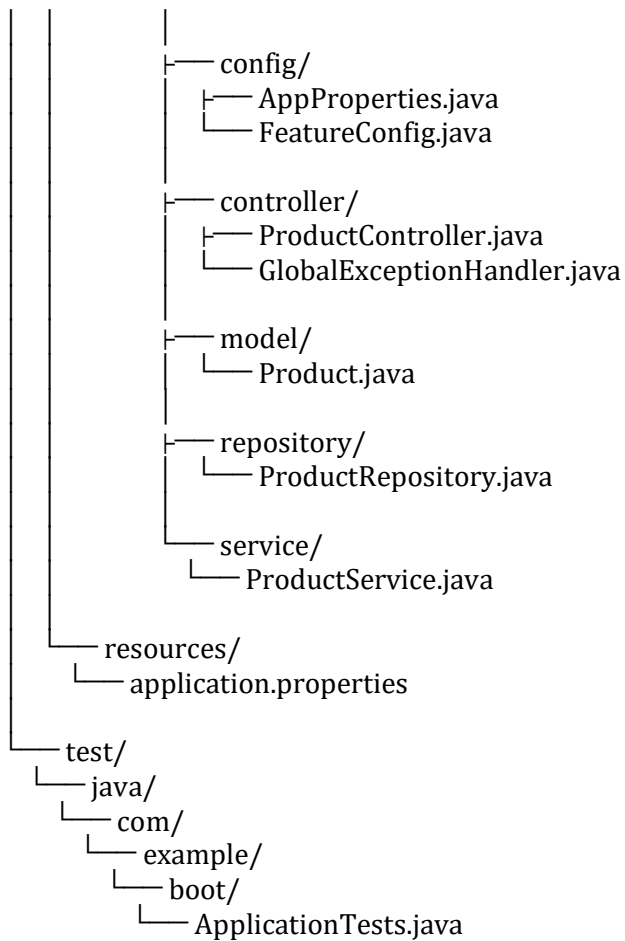
PROJECT 2 — Spring Boot 4.1

Spring Boot 4.1.0 was officially released on June 10, 2026 and is available from Maven Central.

[Home](#)

14. Directory structure

```
spring-boot-4-annotation-sequence/  
├── pom.xml  
├── README.md  
└── src/  
    ├── main/  
    │   ├── java/  
    │   │   ├── com/  
    │   │   │   ├── example/  
    │   │   │   │   └── boot/  
    │   │   │       ├── Application.java  
    │   │   │       └── SpringBootApplication.java
```



15. pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <parent>

    <groupId>
      org.springframework.boot
    </groupId>
```

```
<artifactId>
  spring-boot-starter-parent
</artifactId>

<version>4.1.0</version>

<relativePath/>
</parent>

<groupId>
  com.example
</groupId>

<artifactId>
  spring-boot-4-annotation-sequence
</artifactId>

<version>
  1.0.0
</version>

<name>
  spring-boot-4-annotation-sequence
</name>

<properties>

  <java.version>
    17
  </java.version>

</properties>

<dependencies>

  <!-- Spring MVC / Web -->
  <dependency>

    <groupId>
      org.springframework.boot
    </groupId>
```

```
        <artifactId>
            spring-boot-starter-web
        </artifactId>

    </dependency>

    <!-- Validation -->
    <dependency>

        <groupId>
            org.springframework.boot
        </groupId>

        <artifactId>
            spring-boot-starter-validation
        </artifactId>

    </dependency>

    <!-- Testing -->
    <dependency>

        <groupId>
            org.springframework.boot
        </groupId>

        <artifactId>
            spring-boot-starter-test
        </artifactId>

        <scope>
            test
        </scope>

    </dependency>

</dependencies>

<build>

    <plugins>

        <plugin>

            <groupId>
                org.springframework.boot
            </groupId>
```

```
        <artifactId>
            spring-boot-maven-plugin
        </artifactId>

    </plugin>

</plugins>

</build>

</project>
```

16. application.properties

```
spring.application.name=Spring Boot Annotation Demo

server.port=8081

app.name=Spring Boot Annotation Demo

app.version=1.0

app.feature.audit=true
```

17. Application.java

```
package com.example.boot;

import org.springframework.boot.autoconfigure.SpringBootApplication;

import org.springframework.boot.context.properties.ConfigurationPropertiesScan;

@SpringBootApplication
@ConfigurationPropertiesScan
public class Application {

}
```

This is the **main Spring Boot configuration class**.

The important point is that:

```
@SpringBootApplication
```

represents the central Boot application configuration.

Conceptually it brings together the mechanisms needed for:

Spring configuration

+

Component scanning

+

Spring Boot auto-configuration

18. SpringBootApplication.java

```
package com.example.boot;

import org.springframework.boot.SpringApplication;

public class SpringBootApplication {

    public static void main(String[] args) {

        SpringApplication.run(
            Application.class,
            args
        );
    }
}
```

19. AppProperties.java

```
package com.example.boot.config;

import org.springframework.boot.context.properties.ConfigurationProperties;

@ConfigurationProperties(prefix = "app")
public record AppProperties(

    String name,

    String version,

    Feature feature

) {
```

```
public record Feature(  
    boolean audit  
) {  
    }  
}
```

This maps:

```
app.name=Spring Boot Annotation Demo  
app.version=1.0  
app.feature.audit=true
```

into:

AppProperties

20. FeatureConfig.java

```
package com.example.boot.config;  
  
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;  
  
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.Configuration;  
import org.springframework.context.annotation.Profile;  
  
@Configuration  
public class FeatureConfig {  
  
    @Bean  
  
    @ConditionalOnProperty(  
        prefix = "app.feature",  
        name = "audit",  
        havingValue = "true"  
    )  
    public AuditLogger auditLogger() {  
        return new AuditLogger();  
    }  
}
```

```

    }

    @Bean

    @Profile("dev")
    public String devMarker() {

        return "dev";
    }

    public static class AuditLogger {

        public void log(
            String message
        ) {

            System.out.println(
                "[AUDIT] " + message
            );
        }
    }
}

```

21. Product.java

```

package com.example.boot.model;

import jakarta.validation.constraints.NotBlank;

import jakarta.validation.constraints.Positive;

public record Product(

    long id,

    @NotBlank(
        message = "name is required"
    )
    String name,

    @Positive(
        message = "price must be positive"
    )
    double price

```

```
} {  
}
```

22. ProductRepository.java

```
package com.example.boot.repository;  
  
import com.example.boot.model.Product;  
  
import org.springframework.stereotype.Repository;  
  
import java.util.ArrayList;  
  
import java.util.List;  
  
import java.util.concurrent.atomic.AtomicLong;  
  
@Repository  
public class ProductRepository {  
  
    private final AtomicLong sequence =  
        new AtomicLong(2);  
  
    private final List<Product> products =  
        new ArrayList<>(  
            List.of(  
                new Product(  
                    1L,  
                    "Laptop",  
                    75000.0  
                ),  
                new Product(  
                    2L,  
                    "Monitor",  
                    25000.0  
                )  
            )  
        );
```

```

public List<Product> findAll() {

    return List.copyOf(
        products
    );
}

public Product save(
    Product product
){

    Product saved =

        new Product(

            sequence.incrementAndGet(),

            product.name(),

            product.price()

        );

    products.add(saved);

    return saved;
}

public Product findById(
    long id
){

    return products

        .stream()

        .filter(
            product ->
                product.id() == id
        )

        .findFirst()

        .orElseThrow(

            () ->

```

```

        new ProductNotFoundException(
            id
        )
    );
}

public static class ProductNotFoundException
    extends RuntimeException {

    public ProductNotFoundException(
        long id
    ) {
        super(
            "Product not found: " + id
        );
    }
}
}

```

23. ProductService.java

```

package com.example.boot.service;

import com.example.boot.config.AppProperties;
import com.example.boot.config.FeatureConfig.AuditLogger;
import com.example.boot.model.Product;
import com.example.boot.repository.ProductRepository;
import org.springframework.stereotype.Service;
import org.springframework.validation.annotation.Validated;
import java.util.List;

@Service
@Validated
public class ProductService {

    private final ProductRepository repository;

```

```
private final AppProperties properties;
```

```
private final AuditLogger auditLogger;
```

```
public ProductService(
```

```
    ProductRepository repository,
```

```
    AppProperties properties,
```

```
    AuditLogger auditLogger
```

```
) {
```

```
    this.repository =  
        repository;
```

```
    this.properties =  
        properties;
```

```
    this.auditLogger =  
        auditLogger;
```

```
}
```

```
public List<Product> findAll() {
```

```
    return repository.findAll();  
}
```

```
public Product findById(  
    long id
```

```
) {
```

```
    return repository.findById(id);  
}
```

```
public Product create(  
    Product product
```

```
) {
```

```
    if (  
        properties.feature().audit()
```

```
) {
```

```
        auditLogger.log(
            "Creating product: "
            + product.name()
        );
    }

    return repository.save(
        product
    );
}
}
```

24. ProductController.java

```
package com.example.boot.controller;

import com.example.boot.model.Product;
import com.example.boot.service.ProductService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService service;

    public ProductController(
        ProductService service
    ) {

        this.service = service;
    }

    @GetMapping
```

```

public List<Product> findAll() {
    return service.findAll();
}

@GetMapping("/{id}")
public Product findById(

    @PathVariable
    long id

) {
    return service.findById(id);
}

@PostMapping

@ResponseStatus(
    HttpStatus.CREATED
)

public Product create(

    @Valid

    @RequestBody

    Product product

) {
    return service.create(
        product
    );
}
}

```

25. GlobalExceptionHandler.java

```

package com.example.boot.controller;

import com.example.boot.repository.ProductRepository.ProductNotFoundException;
import org.springframework.http.HttpStatus;

```

```

import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler({
        ProductNotFoundException.class
    })

    @ResponseStatus(
        HttpStatus.NOT_FOUND
    )

    public Map<String, Object> handleNotFound(

        ProductNotFoundException ex
    ) {

        return Map.of(

            "error",
            ex.getMessage(),

            "status",
            404

        );
    }
}

```

26. ApplicationTests.java

```

package com.example.boot;

import org.junit.jupiter.api.Test;

import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class ApplicationTests {

    @Test
    void contextLoads() {

```

```
}  
}
```

27. Run Spring Boot project

cd spring-boot-4-annotation-sequence

Compile and test:

mvn clean test

Run:

mvn spring-boot:run

Application:

<http://localhost:8081/api/products>

28. Test the REST API

GET all products

GET <http://localhost:8081/api/products>

Expected response:

```
[  
  {  
    "id": 1,  
    "name": "Laptop",  
    "price": 75000.0  
  },  
  {  
    "id": 2,  
    "name": "Monitor",  
    "price": 25000.0  
  }  
]
```

GET one product

GET http://localhost:8081/api/products/1

Response:

```
{
  "id": 1,
  "name": "Laptop",
  "price": 75000.0
}
```

POST a product

POST http://localhost:8081/api/products

Content-Type: application/json

Body:

```
{
  "id": 0,
  "name": "Keyboard",
  "price": 3500
}
```

Response:

```
{
  "id": 3,
  "name": "Keyboard",
  "price": 3500.0
}
```

GET nonexistent product

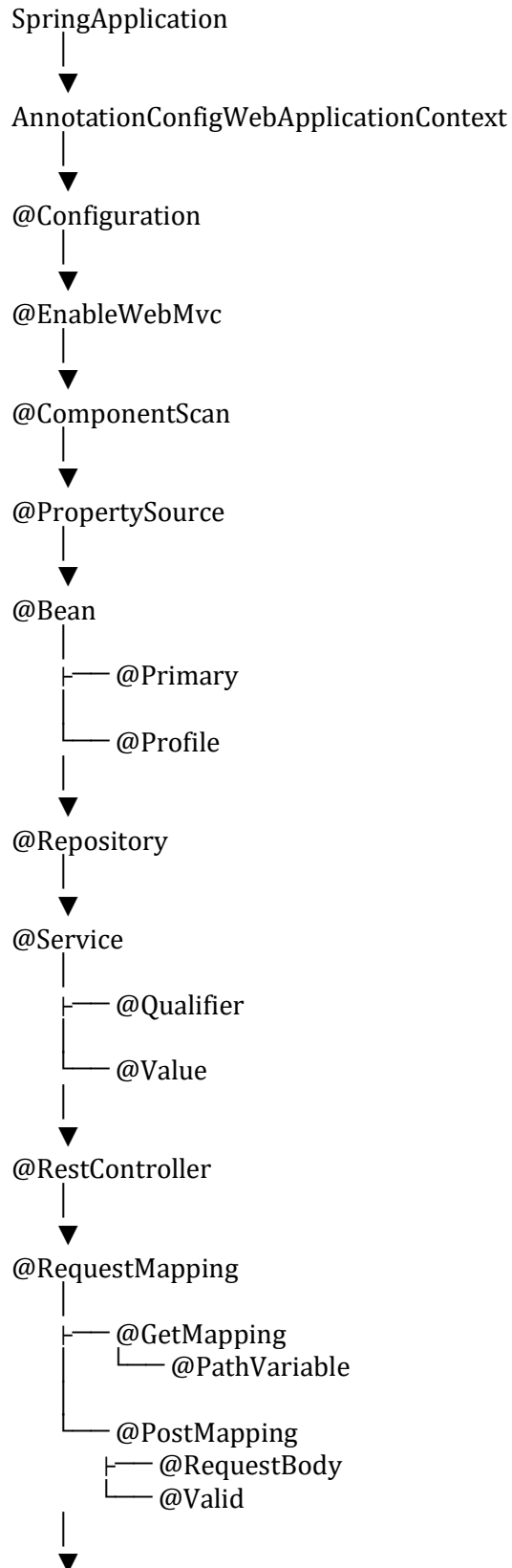
GET http://localhost:8081/api/products/999

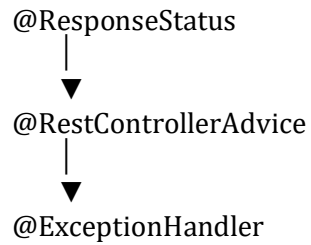
Response:

```
{
  "error": "Product not found: 999",
  "status": 404
}
```

29. Complete annotation flow — Spring Framework

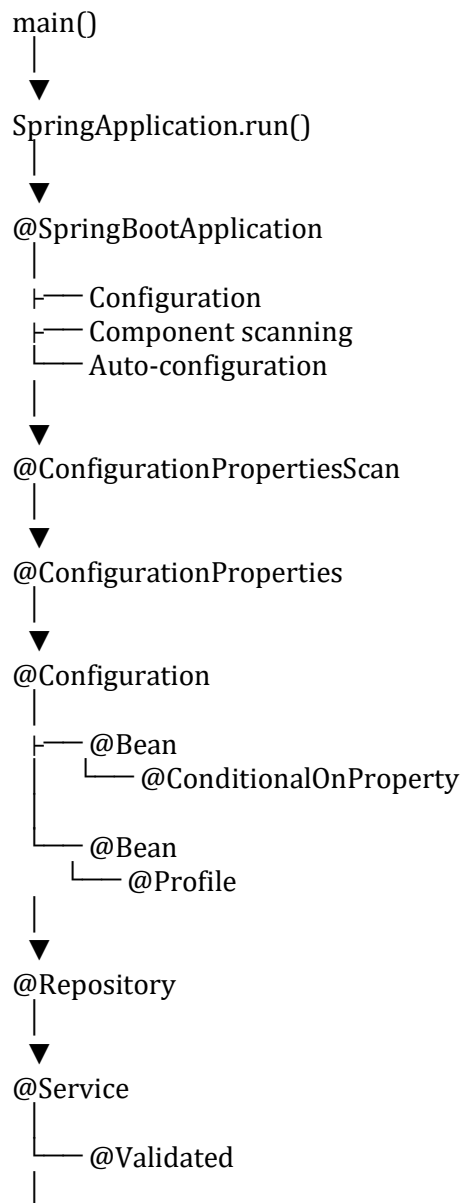
The learning sequence in Project 1 is:

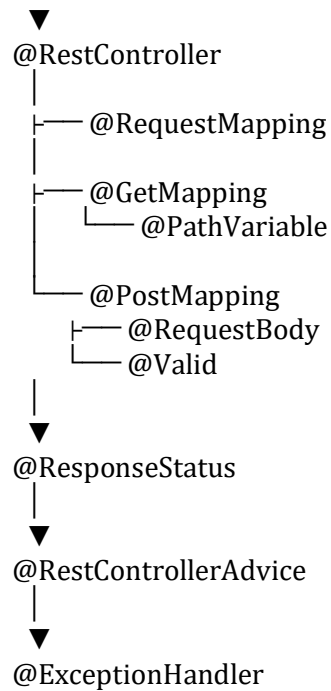




30. Complete annotation flow — Spring Boot

Project 2 follows:





Important Spring vs Spring Boot difference

Area	Spring Framework project	Spring Boot project
Entry point	Custom Java main	SpringApplication.run()
Container	Explicitly created	Boot creates/configures it
Web server	Manually embedded/configured	Auto-configured
MVC	@EnableWebMvc	Starter + auto-configuration
Component scanning	Explicit @ComponentScan	Included through @SpringBootApplication
Configuration	@Configuration	@SpringBootApplication + @Configuration
Properties	@PropertySource + @Value	application.properties + @ConfigurationProperties
Dependency management	Explicit versions	Boot dependency management

Server port	Explicit server setup	server.port
Conditional configuration	Basic Spring conditions	Boot conditions such as @ConditionalOnProperty
Deployment style	More manual	Convention over configuration
Complexity	Higher	Lower
Auto-configuration	No	Yes
Embedded server setup	Manual	Automatic
Typical entry point	SpringApplication custom class	SpringApplication.run()
Best for learning	Spring internals	Production-style application development

Spring's Java configuration model explicitly centers on @Configuration and @Bean. [Home](#) Spring Boot 4.1 adds Boot-specific conveniences and auto-configuration on top of the Spring Framework. The 4.1 GA release also includes dependency and infrastructure updates. [Home](#)

One caveat: the code above is intentionally a compact, self-contained learning application. It uses an in-memory List instead of a database so that the annotation lifecycle is visible without introducing JPA, transactions, migrations, security, or external infrastructure.